

OpenAI 大模型 开发与应用实践

彭靖田

Google Developer Experts
LangChain Developer



目录

- OpenAI 大模型开发指南
 - OpenAI 语言模型总览
 - OpenAI GPT-4, GPT-3.5, GPT-3, Moderation
 - OpenAI Token 计费与计算
- OpenAI API 入门与实战
 - OpenAI Models API
 - OpenAI Completions API
 - OpenAI Chat Completions API
 - Completions vs Chat Completions
- OpenAI 大模型应用实践
 - 文本内容补全初探 (Text Completion)
 - 聊天机器人初探 (Chat Completion)

OpenAI 大模型开发指南

OpenAI 语言模型总览

MODELS	DESCRIPTION
Embeddings	A set of models that can convert text into a numerical form
GPT-4	A set of models that improve on GPT-3.5 and can understand as well as generate natural language or code
GPT-3.5	A set of models that improve on GPT-3 and can understand as well as generate natural language or code
GPT-3	A set of models that can understand and generate natural language
Moderation	A fine-tuned model that can detect whether text may be sensitive or unsafe
Deprecated	A full list of models that have been deprecated

Model: GPT-4

i GPT-4 is currently accessible to those who have made at least one successful payment through our developer platform.

GPT-4 is a large multimodal model (accepting text inputs and emitting text outputs today, with image inputs coming in the future) that can solve difficult problems with greater accuracy than any of our previous models, thanks to its broader general knowledge and advanced reasoning capabilities. Like `gpt-3.5-turbo`, GPT-4 is optimized for chat but works well for traditional completions tasks both using the [Chat Completions API](#). Learn how to use GPT-4 in our [GPT guide](#).

LATEST MODEL	DESCRIPTION	MAX TOKENS	TRAINING DATA
<code>gpt-4</code>	More capable than any GPT-3.5 model, able to do more complex tasks, and optimized for chat. Will be updated with our latest model iteration 2 weeks after it is released.	8,192 tokens	Up to Sep 2021
<code>gpt-4-0613</code>	Snapshot of <code>gpt-4</code> from June 13th 2023 with function calling data. Unlike <code>gpt-4</code> , this model will not receive updates, and will be deprecated 3 months after a new version is released.	8,192 tokens	Up to Sep 2021
<code>gpt-4-32k</code>	Same capabilities as the base <code>gpt-4</code> mode but with 4x the context length. Will be updated with our latest model iteration.	32,768 tokens	Up to Sep 2021
<code>gpt-4-32k-0613</code>	Snapshot of <code>gpt-4-32</code> from June 13th 2023. Unlike <code>gpt-4-32k</code> , this model will not receive updates, and will be deprecated 3 months after a new version is released.	32,768 tokens	Up to Sep 2021

For many basic tasks, the difference between GPT-4 and GPT-3.5 models is not significant. However, in more complex reasoning situations, GPT-4 is much more capable than any of our previous models.

Model: GPT-3.5

Our most capable and cost effective model in the GPT-3.5 family is **gpt-3.5-turbo** which has been optimized for chat but works well for traditional completions tasks as well.

gpt-3.5-turbo	Most capable GPT-3.5 model and optimized for chat at 1/10th the cost of text-davinci-003. Will be updated with our latest model iteration 2 weeks after it is released.	4,096 tokens	Up to Sep 2021
gpt-3.5-turbo-16k	Same capabilities as the standard gpt-3.5-turbo model but with 4 times the context.	16,384 tokens	Up to Sep 2021
gpt-3.5-turbo-0613	Snapshot of gpt-3.5-turbo from June 13th 2023 with function calling data. Unlike gpt-3.5-turbo, this model will not receive updates, and will be deprecated 3 months after a new version is released.	4,096 tokens	Up to Sep 2021
gpt-3.5-turbo-16k-0613	Snapshot of gpt-3.5-turbo-16k from June 13th 2023. Unlike gpt-3.5-turbo-16k, this model will not receive	16,384 tokens	Up to Sep 2021

text-davinci-003	(Legacy)	Can do any language task with better quality, longer output, and consistent instruction-following than the curie, babbage, or ada models. Also supports some additional features such as inserting text.	4,097 tokens	Up to Jun 2021
text-davinci-002	(Legacy)	Similar capabilities to text-davinci-003 but trained with supervised fine-tuning instead of reinforcement learning	4,097 tokens	Up to Jun 2021
code-davinci-002	(Legacy)	Optimized for code-completion tasks	8,001 tokens	Up to Jun 2021

We recommend using `gpt-3.5-turbo` over the other GPT-3.5 models because of its lower cost and improved performance.

i OpenAI models are non-deterministic, meaning that identical inputs can yield different outputs. Setting `temperature` to 0 will make the outputs mostly deterministic, but a small amount of variability may remain.

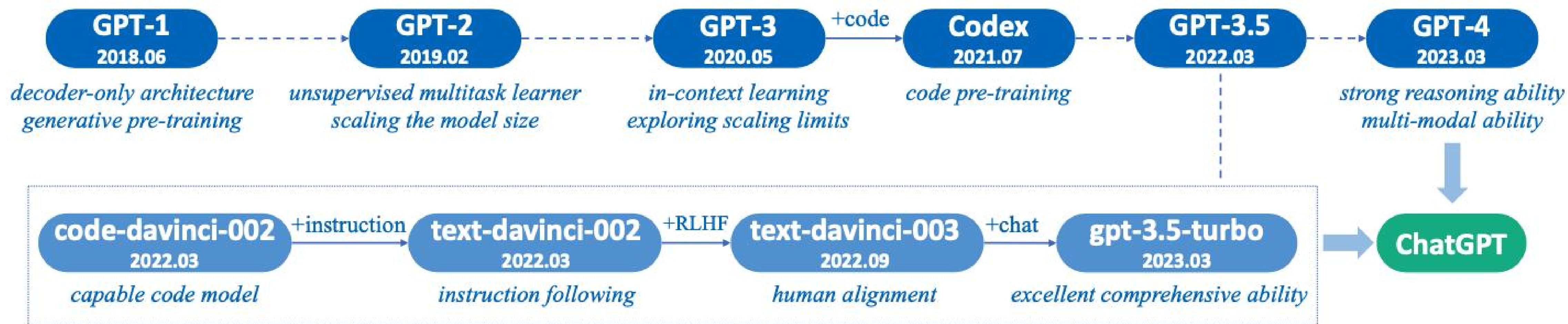


Fig. 3: A brief illustration for the technical evolution of GPT-series models. We plot this figure mainly based on the papers, blog articles and official APIs from OpenAI. Here, *solid lines* denote that there exists an explicit evidence (e.g., the official statement that a new model is developed based on a base model) on the evolution path between two models, while *dashed lines* denote a relatively weaker evolution relation.



GPT-3 Legacy

GPT-3 models can understand and generate natural language. These models were superceded by the more powerful GPT-3.5 generation models. However, the original GPT-3 base models (davinci, curie, ada, and babbage) are currently the only models that are available to fine-tune.

LATEST MODEL	DESCRIPTION	MAX TOKENS	TRAINING DATA
text-curie-001	Very capable, faster and lower cost than Davinci.	2,049 tokens	Up to Oct 2019
text-babbage-001	Capable of straightforward tasks, very fast, and lower cost.	2,049 tokens	Up to Oct 2019
text-ada-001	Capable of very simple tasks, usually the fastest model in the GPT-3 series, and lowest cost.	2,049 tokens	Up to Oct 2019
davinci	Most capable GPT-3 model. Can do any task the other models can do, often with higher quality.	2,049 tokens	Up to Oct 2019
curie	Very capable, but faster and lower cost than Davinci.	2,049 tokens	Up to Oct 2019
babbage	Capable of straightforward tasks, very fast, and lower cost.	2,049 tokens	Up to Oct 2019
ada	Capable of very simple tasks, usually the fastest model in the GPT-3 series, and lowest cost.	2,049 tokens	Up to Oct 2019

Benchmark of GPT-3

Ada, Babbage, Curie和Davinci分别紧密对应着3.5亿、13亿、67亿和1750亿。

Model	LAMBADA ppl ↓	LAMBADA acc ↑	Winogrande ↑	Hellaswag ↑	PIQA ↑
GPT-3-124M	18.6	42.7%	52.0%	33.7%	64.6%
GPT-3-350M	9.09	54.3%	52.1%	43.6%	70.2%
Ada	9.95	51.6%	52.9%	43.4%	70.5%
GPT-3-760M	6.53	60.4%	57.4%	51.0%	72.9%
GPT-3-1.3B	5.44	63.6%	58.7%	54.7%	75.1%
Babbage	5.58	62.4%	59.0%	54.5%	75.5%
GPT-3-2.7B	4.60	67.1%	62.3%	62.8%	75.6%
GPT-3-6.7B	4.00	70.3%	64.5%	67.4%	78.0%
Curie	4.00	68.5%	65.6%	68.5%	77.9%
GPT-3-13B	3.56	72.5%	67.9%	70.9%	78.5%
GPT-3-175B	3.00	76.2%	70.2%	78.9%	81.0%
Davinci	2.97	74.8%	70.2%	78.1%	80.4%
All GPT-3 figures are from the GPT-3 paper ; all API figures are computed using eval harness					

Fine-tuning of GPT-3

- i** On July 6, 2023, we **announced** the deprecation of ada, babbage, curie and davinci models. These models, including fine-tuned versions, **will be turned off on January 4, 2024.** We are actively working on enabling fine-tuning for upgraded base GPT-3 models as well as GPT-3.5 Turbo and GPT-4, we recommend waiting for those new options to be available rather than fine-tuning based off of the soon to be deprecated models.

Fine-tuning lets you get more out of the models available through the API by providing:

- 1 Higher quality results than prompt design
- 2 Ability to train on more examples than can fit in a prompt
- 3 Token savings due to shorter prompts
- 4 Lower latency requests

GPT-3 has been pre-trained on a vast amount of text from the open internet. When given a prompt with just a few examples, it can often intuit what task you are trying to perform and generate a plausible completion. This is often called "few-shot learning."

Fine-tuning improves on few-shot learning by training on many more examples than can fit in the prompt, letting you achieve better results on a wide number of tasks. **Once a model has been fine-tuned, you won't need to provide examples in the prompt anymore.** This saves costs and enables lower-latency requests.

The Moderation models are designed to check whether content complies with OpenAI's [usage policies](#). The models provide classification capabilities that look for content in the following categories: hate, hate/threatening, self-harm, sexual, sexual/minors, violence, and violence/graphic. You can find out more in our [moderation guide](#).

Moderation models take in an arbitrary sized input that is automatically broken up to fit the models specific context window.

MODEL	DESCRIPTION
text-moderation-latest	Most capable moderation model. Accuracy will be slightly higher than the stable model.
text-moderation-stable	Almost as capable as the latest model, but slightly older.

Model: Moderation

Model: Moderation

- The moderation endpoint is **free to use** when monitoring the inputs and outputs of OpenAI APIs.
- We currently **disallow other use cases**.
- Accuracy may be **lower on longer pieces of text**.
- For higher accuracy, try splitting long pieces of text into smaller chunks each **less than 2,000 characters**.

CATEGORY	DESCRIPTION
hate	Content that expresses, incites, or promotes hate based on race, gender, ethnicity, religion, nationality, sexual orientation, disability status, or caste. Hateful content aimed at non-protected groups (e.g., chess players) is harrassment.
hate/threatening	Hateful content that also includes violence or serious harm towards the targeted group based on race, gender, ethnicity, religion, nationality, sexual orientation, disability status, or caste.
harassment	Content that expresses, incites, or promotes harassing language towards any target.
harassment/threatening	Harassment content that also includes violence or serious harm towards any target.
self-harm	Content that promotes, encourages, or depicts acts of self-harm, such as suicide, cutting, and eating disorders.
self-harm/intent	Content where the speaker expresses that they are engaging or intend to engage in acts of self-harm, such as suicide, cutting, and eating disorders.
self-harm/instructions	Content that encourages performing acts of self-harm, such as suicide, cutting, and eating disorders, or that gives instructions or advice on how to commit such acts.
sexual	Content meant to arouse sexual excitement, such as the description of sexual activity, or that promotes sexual services (excluding sex education and wellness).
sexual/minors	Sexual content that includes an individual who is under 18 years old.
violence	Content that depicts death, violence, or physical injury.
violence/graphic	Content that depicts death, violence, or physical injury in graphic detail.

Token Pricing & Counting

Pricing: Language Models

GPT-4 With broad general knowledge and domain expertise, GPT-4 can follow complex instructions in natural language and solve difficult problems with accuracy. Learn about GPT-4		
Model	Input	Output
8K context	\$0.03 / 1K tokens	\$0.06 / 1K tokens
32K context	\$0.06 / 1K tokens	\$0.12 / 1K tokens
GPT-3.5 Turbo GPT-3.5 Turbo is optimized for dialogue. Learn about GPT-3.5 Turbo ↗		
Model	Input	Output
4K context	\$0.0015 / 1K tokens	\$0.002 / 1K tokens
16K context	\$0.003 / 1K tokens	\$0.004 / 1K tokens

Fine-tuning models Create your own custom models by fine-tuning our base models with your training data. Once you fine-tune a model, you'll be billed only for the tokens you use in requests to that model. Learn about fine-tuning ↗		
Model	Training	Usage
Ada	\$0.0004 / 1K tokens	\$0.0016 / 1K tokens
Babbage	\$0.0006 / 1K tokens	\$0.0024 / 1K tokens
Curie	\$0.0030 / 1K tokens	\$0.0120 / 1K tokens
Davinci	\$0.0300 / 1K tokens	\$0.1200 / 1K tokens

Embedding models Build advanced search, clustering, topic modeling, and classification functionality with our embeddings offering. Learn about embeddings ↗	
Model	Usage
Ada v2	\$0.0001 / 1K tokens

每个模型都具有不同的功能和价格。价格是按照每1000个 Token 计算的。您可以将Token视为单词的组成部分，其中1000个Token大约相当于750个单词。（这段文字包含35个Token）

每个模型都具有不同的功能和价格。价格是按照每1000个 Token 计算的。您可以将Token视为单词的组成部分，其中1000个Token大约相当于750个单词。（这段文字包含35个Token）

GPT-3

Codex

Multiple models, each with different capabilities and price points.
Prices are per 1000 tokens. You can think of tokens as pieces of words,
where 1000 tokens is about 750 words.

Clear

Show example

Tokens

35

Characters

177

Multiple models, each with different capabilities and price points.
Prices are per 1000 tokens. You can think of tokens as pieces of words,
where 1000 tokens is about 750 words.

TEXT

TOKEN IDS

GPT-3

Codex

Multiple models, each with different capabilities and price points.
Prices are per 1000 tokens. You can think of tokens as pieces of words,
where 1000 tokens is about 750 words.

Clear

Show example

Tokens

35

Characters

177

[31217, 4981, 11, 1123, 351, 1180, 9889, 290, 2756, 2173, 13, 29431, 389,
583, 8576, 16326, 13, 921, 460, 892, 286, 16326, 355, 5207, 286, 2456,
11, 810, 8576, 16326, 318, 546, 19683, 2456, 13]

TEXT

TOKEN IDS

tiktoken: a fast BPE(Byte pair encoding) Tokenizer



tiktoken is a fast **BPE** tokeniser for use with OpenAI's models.

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")
assert enc.decode(enc.encode("hello world")) == "hello world"

# To get the tokeniser corresponding to a specific model in the OpenAI API:
enc = tiktoken.encoding_for_model("gpt-4")
```

The open source version of **tiktoken** can be installed from PyPI:

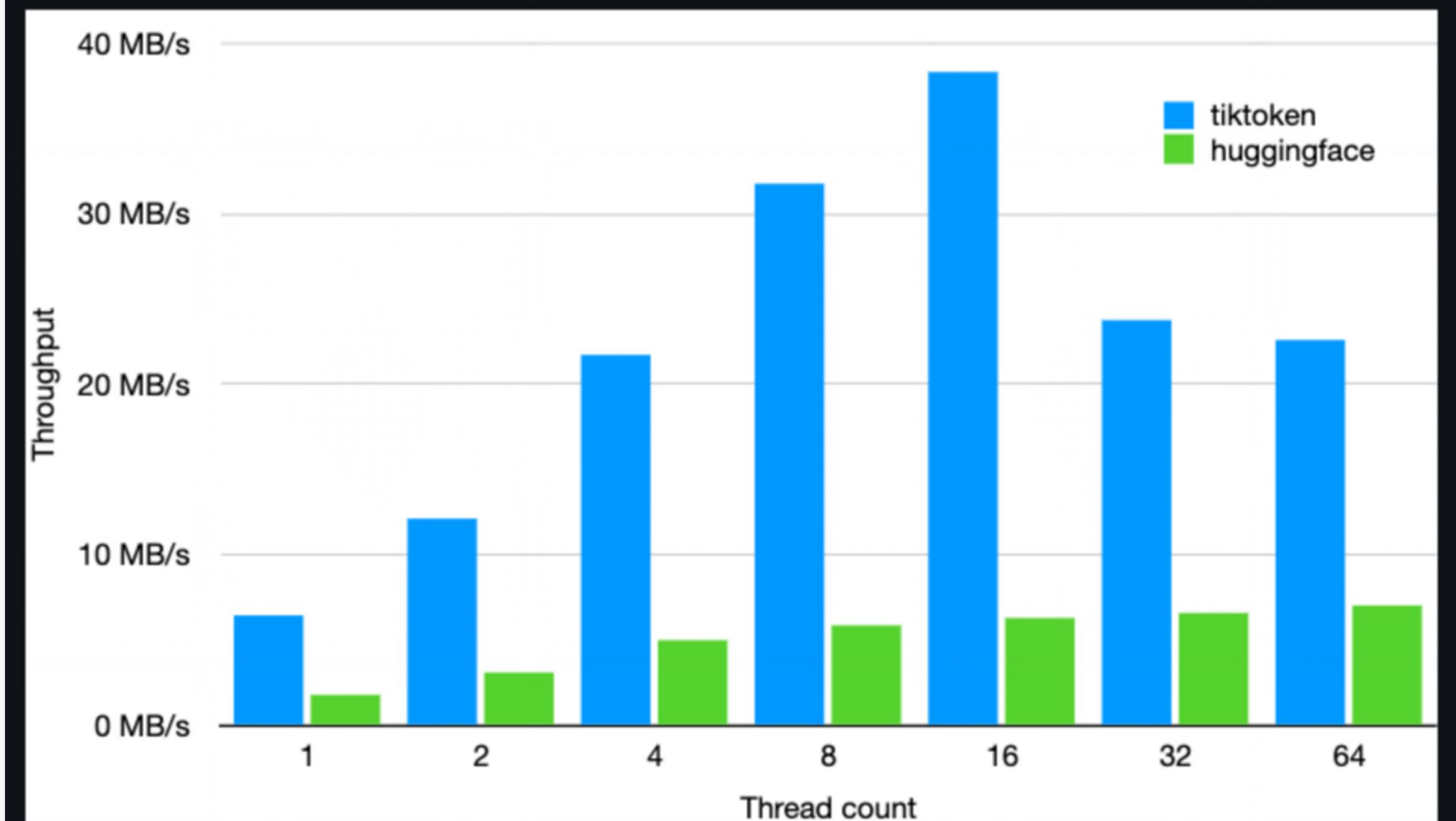
```
pip install tiktoken
```

The tokeniser API is documented in **tiktoken/core.py**.

Example code using **tiktoken** can be found in the [OpenAI Cookbook](#).

Performance

tiktoken is between 3-6x faster than a comparable open source tokeniser:



Performance measured on 1GB of text using the GPT-2 tokeniser, using **GPT2TokenizerFast** from **tokenizers==0.13.2**, **transformers==4.24.0** and **tiktoken==0.2.0**.

OpenAI API 入门与实战

Models API

List Models

```
GET https://api.openai.com/v1/models
```

Lists the currently available models, and provides basic information about each one such as the owner and availability.

Example request

python  Copy

```
1 import os
2 import openai
3 openai.api_key = os.getenv("OPENAI_API_KEY")
4 openai.Model.list()
```

Response

 Copy

```
1 {
2   "data": [
3     {
4       "id": "model-id-0",
5       "object": "model",
6       "owned_by": "organization-owner",
7       "permission": [...]
8     },
9     {
10      "id": "model-id-1",
11      "object": "model",
12      "owned_by": "organization-owner",
13      "permission": [...]
14    },
15    {
16      "id": "model-id-2",
17      "object": "model",
18      "owned_by": "openai",
19      "permission": [...]
20    },
21  ],
22  "object": "list"
23 }
```

Retrieve Model

Example req... text-davinci-003 ▾ python ▾ Copy

```
1 import os
2 import openai
3 openai.api_key = os.getenv("OPENAI_API_KEY")
4 openai.Model.retrieve("text-davinci-003")
```

Response text-davinci-003 ▾ Copy

```
1 {
2   "id": "text-davinci-003",
3   "object": "model",
4   "owned_by": "openai",
5   "permission": [...]
6 }
```

GET https://api.openai.com/v1/models/{model}

Retrieves a model instance, providing basic information about the model such as the owner and permissioning.

Path parameters

model string **Required**

The ID of the model to use for this request

OpenAI 模型弃用计划：从 2024 年 1 月 4 日开始，GPT-3 及其衍生模型都将不再可用，并将被新的 "GPT-3 "基础模型所取代，新的模型计算效率会更高。旧的嵌入模型（如 text-search-davinci-doc-001）的用户也需要在 2024 年 1 月 4 日前迁移到 text-embedding-ada-002。

Older model	New model
ada	ada-002
babbage	babbage-002
curie	curie-002
davinci	davinci-002
davinci-instruct-beta	
curie-instruct-beta	
text-ada-001	
text-babbage-001	
text-curie-001	gpt-3.5-turbo-instruct
text-davinci-001	
text-davinci-002	
text-davinci-003	

Older model	New model
code-search-ada-code-001	
code-search-ada-text-001	
code-search-babbage-code-001	
code-search-babbage-text-001	
text-search-ada-doc-001	
text-search-ada-query-001	
text-search-babbage-doc-001	
text-search-babbage-query-001	text-embedding-ada-002
text-search-curie-doc-001	
text-search-curie-query-001	
text-search-davinci-doc-001	
text-search-davinci-query-001	
text-similarity-ada-001	
text-similarity-babbage-001	
text-similarity-curie-001	
text-similarity-davinci-001	

Completions API

Completions API

文本补全API: 给定一个Prompt, 模型将返回一个或多个文本生成结果, 并且还可以返回每个位置上备选tokens的概率。

POST https://api.openai.com/v1/completions

Creates a completion for the provided prompt and parameters.

Request body

model string Required

ID of the model to use. You can use the [List models](#) API to see all of your available models, or see our [Model overview](#) for descriptions of them.

prompt string or array Required

The prompt(s) to generate completions for, encoded as a string, array of strings, array of tokens, or array of token arrays.

Note that <|endoftext|> is the document separator that the model sees during training, so if a prompt is not specified the model will generate as if from the beginning of a new document.

suffix string Optional Defaults to null

The suffix that comes after a completion of inserted text.

max_tokens integer Optional Defaults to 16

The maximum number of **tokens** to generate in the completion.

The token count of your prompt plus `max_tokens` cannot exceed the model's context length. [Example Python code](#) for counting tokens.

temperature number Optional Defaults to 1

What sampling temperature to use, between 0 and 2. Higher values like 0.8 will make the output more random, while lower values like 0.2 will make it more focused and deterministic.

We generally recommend altering this or `top_p` but not both.

top_p number Optional Defaults to 1

An alternative to sampling with temperature, called nucleus sampling, where the model considers the results of the tokens with top_p probability mass. So 0.1 means only the tokens comprising the top 10% probability mass are considered.

We generally recommend altering this or `temperature` but not both.

Example req... text-davinci-003 python Copy

```
1 import os
2 import openai
3 openai.api_key = os.getenv("OPENAI_API_KEY")
4 openai.Completion.create(
5     model="text-davinci-003",
6     prompt="Say this is a test",
7     max_tokens=7,
8     temperature=0
9 )
```

Parameters text-davinci-003 Copy

```
1 {
2     "model": "text-davinci-003",
3     "prompt": "Say this is a test",
4     "max_tokens": 7,
5     "temperature": 0,
6     "top_p": 1,
7     "n": 1,
8     "stream": false,
9     "logprobs": null,
10    "stop": "\n"
11 }
```

Response text-davinci-003 Copy

```
1 {
2     "id": "cmpl-uqkv1QyYK7bGYrRHQ0eXlWi7",
3     "object": "text_completion",
4     "created": 1589478378,
5     "model": "text-davinci-003",
6     "choices": [
7         {
8             "text": "\n\nThis is indeed a test",
9             "index": 0,
10            "logprobs": null,
11            "finish_reason": "length"
12        }
13    ],
14    "usage": {
15        "prompt_tokens": 5,
16        "completion_tokens": 7,
17        "total_tokens": 12
18    }
19 }
```

n integer Optional Defaults to 1

How many completions to generate for each prompt.

Note: Because this parameter generates many completions, it can quickly consume your token quota. Use carefully and ensure that you have reasonable settings for `max_tokens` and `stop`.

stream boolean Optional Defaults to false

Whether to stream back partial progress. If set, tokens will be sent as data-only **server-sent events** as they become available, with the stream terminated by a `data: [DONE]` message.

[Example Python code.](#)

logprobs integer Optional Defaults to null

Include the log probabilities on the `logprobs` most likely tokens, as well the chosen tokens. For example, if `logprobs` is 5, the API will return a list of the 5 most likely tokens. The API will always return the `logprob` of the sampled token, so there may be up to `logprobs+1` elements in the response.

The maximum value for `logprobs` is 5.

echo boolean Optional Defaults to false

Echo back the prompt in addition to the completion

stop string or array Optional Defaults to null

Up to 4 sequences where the API will stop generating further tokens. The returned text will not contain the stop sequence.

presence_penalty number Optional Defaults to 0

Number between -2.0 and 2.0. Positive values penalize new tokens based on whether they appear in the text so far, increasing the model's likelihood to talk about new topics.

[See more information about frequency and presence penalties.](#)

frequency_penalty number Optional Defaults to 0

Number between -2.0 and 2.0. Positive values penalize new tokens based on their existing frequency in the text so far, decreasing the model's likelihood to repeat the same line verbatim.

[See more information about frequency and presence penalties.](#)

Completions API: Inserting Text

To illustrate how suffix context effects generated text, consider the prompt, "Today I decided to make a big change." There's many ways one could imagine completing the sentence. But if we now supply the ending of the story: "I've gotten many compliments on my new hair!", the intended completion becomes clear.

I went to college at Boston University. After getting my degree, I decided to make a change. A big change!

I packed my bags and moved to the west coast of the United States.

Now, I can't get enough of the Pacific Ocean!

Chat Completions API

Chat Completions API

聊天补全API： 给定一个包含对话的消息列表，返回模型生成的消息作为输出。尽管聊天格式旨在使多轮对话变得简单，但它同样适用于没有任何对话的单轮任务。

POST https://api.openai.com/v1/chat/completions

Creates a model response for the given chat conversation.

Request body

model string Required

ID of the model to use. See the [model endpoint compatibility table](#) for details on which models work with the Chat API.

messages array Required

A list of messages comprising the conversation so far. [Example Python code.](#)

role string Required

The role of the messages author. One of `system`, `user`, `assistant`, or `function`.

content string Required

The contents of the message. `content` is required for all messages, and may be null for assistant messages with function calls.

name string Optional

The name of the author of this message. `name` is required if role is `function`, and it should be the name of the function whose response is in the `content`. May contain a-z, A-Z, 0-9, and underscores, with a maximum length of 64 characters.

function_call object Optional

The name and arguments of a function that should be called, as generated by the model.

functions array Optional

A list of functions the model may generate JSON inputs for.

Example request gpt-3.5-turbo python Copy

```
1 import os
2 import openai
3 openai.api_key = os.getenv("OPENAI_API_KEY")
4
5 completion = openai.ChatCompletion.create(
6     model="gpt-3.5-turbo",
7     messages=[
8         {"role": "system", "content": "You are a"},
9         {"role": "user", "content": "Hello!"}
10    ]
11 )
12
13 print(completion.choices[0].message)
```

Parameters gpt-3.5-turbo Copy

```
1 {
2     "model": "gpt-3.5-turbo",
3     "messages": [{"role": "system", "content": "You are a"},
4                   {"role": "user", "content": "Hello!"}]
5 }
```

Response Copy

```
1 {
2     "id": "chatcmpl-123",
3     "object": "chat.completion",
4     "created": 1677652288,
5     "choices": [{
6         "index": 0,
7         "message": {
8             "role": "assistant",
9             "content": "\n\nHello there, how may I"
10        },
11         "finish_reason": "stop"
12     }],
13     "usage": {
14         "prompt_tokens": 9,
15         "completion_tokens": 12,
16         "total_tokens": 21
17     }
18 }
```

System
Apply the change requested by the user to the code.

User
def fibonacci(num):
 if num <= 1:
 return num
 else:
 return fibonacci(num-1) + fibonacci(num-2)
print(fibonacci(10))

User
Rename the function to fib.

Assistant
def fib(num):
 if num <= 1:
 return num
 else:
 return fib(num-1) + fib(num-2)
print(fib(10))

Completions vs Chat Completions

The chat completions format can be made similar to the completions format by constructing a request using a single user message. For example, one can translate from English to French with the following completions prompt:

Translate the following English text to French: "{text}"

And an equivalent chat prompt would be:

```
[  
  {  
    "role": "user",  
    "content": "Translate the following English text to French: "{text}"",  
  },  
]
```

DEMO

OpenAI 大模型应用实践

课程项目：GitHub openai-quickstart

openai-quickstartPublic

UnpinUnwatch 5Fork 53Star 76

main1 branch0 tags

Go to fileAdd fileCode

DjangoPeng add demo of models and tiktokenbeb1a63 now12 commits

openai_apiadd demo of models and tiktokennow

.gitignoreupdate embedding demo2 days ago

LICENSEInitial commitlast week

README.mdUpdate README.mdlast week

README.md

OpenAI Quickstart

This project is designed as a one-stop learning resource for anyone interested in large language models and their application in Artificial Intelligence Governance and Control (AIGC) scenarios. By providing theoretical foundations, development basics, and hands-on examples, this project offers comprehensive guidance on these cutting-edge topics.

Features

- Theory and Development Basics of Large Language Models:** Deep dive into the inner workings of large language models like GPT-4, including their architecture, training methods, applications, and more.
- AIGC Application Development with LangChain:** Hands-on examples and tutorials using LangChain to develop AIGC applications, demonstrating the practical application of large language models.

Getting Started

You can start by cloning this repository to your local machine:

```
git clone https://github.com/DjangoPeng/openai-quickstart.git
```

About

A comprehensive guide to understanding and implementing large language models with hands-on examples using LangChain for AIGC applications.

Readme

Apache-2.0 license

Activity

76 stars

5 watching

53 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 4

DjangoPengJingtian Peng

FukoHShengxin Huang

wr-fengleiSam Feng

gg-daddyScott Chen

<https://github.com/DjangoPeng/openai-quickstart>

 DjangoPeng

add demo of models and tiktoken

beb1a63 · 1 minute ago

History

Name	Last commit message	Last commit date
..		
FAQ	add faq to each lesson	4 days ago
data	add initial demo of embedding	4 days ago
images	add demo of models and tiktoken	1 minute ago
README-CN.md	add initial demo of embedding	4 days ago
README.md	add initial demo of embedding	4 days ago
count_tokens_with_tiktoken.ipynb	add demo of models and tiktoken	1 minute ago
embedding.ipynb	add demo of models and tiktoken	1 minute ago
models.ipynb	add demo of models and tiktoken	1 minute ago

README.md

OpenAI Quickstart Guide

This project guides you through the initial steps of developing applications using the OpenAI API. It will help you setup your development environment, understand how to use the API, and provide you with a working Jupyter Lab notebook for interactive development.

Getting Started

Here are the steps to get your development environment ready:

DEMO 实战